



ENVIRONMENTAL DATA ANALYTICS: M10 – DATA SCRAPING

Agenda

- Remaining Schedule...
 - Course Wrap up
 - Projects
 - Python for R users
- M10 Data scraping...

Error in 10_Data_Scraping.Rmd



Dates are scraped in the wrong order!

2-A. Average daily withdrawal and maximum day withdrawal by month in million gallons per day (MGD)

Month	# of Days Used	Average Daily Withdrawal (MGD)	Maximum Day Withdrawal (MGD)	Month	# of Days Used	Average Daily Withdrawal (MGD)	Maximum Day Withdrawal (MGD)
Jan	31	.584	1.029	Jul	31	.492	1.045
Feb	29	.647	1.056	Aug	31	.613	1.115
Mar	31	.543	.934	Sep	30	.683	1.096
Apr	30	.371	.978	Oct	31	.650	1.018
May	31	.349	1.146	Nov	30	.535	1.046
Jun	30	.447	1.111	Dec	31	.449	.889

```
137 - ## 1.4 Construct a dataframe from the data & plot the values
138
```

```
139 Now that we've scraped the data we want into our coding environment, the next step is to wrangle these data
    into formats we can work with more easily, i.e. a dataframe. Here, we employ the techniques we've learned
    earlier in this class to do this and then plot the data.
```

```
140 - ```{r create.dataframe.from.scrapings}
```

```
141 #Create a dataframe of withdrawals
```

```
142 df_withdrawals <- data.frame("Month" = c(1,7,2,8,3,9,4,10,5,11,6,12), #<--NOTE: DIFFERENT THAN RECORDING!
```

```
143                               "Year" = rep(2020,12),
```

```
144                               "Avg_Withdrawals_mgd" = as.numeric(avg_withdrawals),
```

```
145                               "Max_Withdrawals_mgd" = as.numeric(max_withdrawals))
```

```
146
```

```
df_withdrawals <- data.frame("Month" = as.vector(rbind(1:6,7:12))).
```

Revision in 10_Data_Scraping.Rmd

- Mapping 2 variables to a function with ``pmap()``

```
377 ▾ ### REVISED VERSION OF ABOVE
378 ▾ ```{r}
379 #Select the last three facilities
380 the_facilities <- tail(the_facility_ids,3)
381 #Create a vector of years
382 the_years <- c(2018,2019,2020)
383
384 #Create a "grid" of input parameters
385 params <- expand.grid(the_year = the_years, the_facility = the_facilities)
386
387 #Map
388 the_df <- pmap(params, scrape.it) %>% bind_rows()
389
390 #Plot
391 ggplot(the_df,aes(x=Date,y=Avg-Withdrawals_mgd,color=Facility_name)) +
392   #geom_line() +
393   geom_smooth(method="loess",se=FALSE) +
394   labs(title = "Water usage data",
395         y="Withdrawal (mgd)",
396         x="Date")
397 ▴ ```
```

- ``expand.grid()`` creates a dataframe of all combinations of inputs
- ``pmap()`` repeats the function for these unique combinations

Scraping data...

- **rvest** package:
 - ▣ **read_html()** → Reads a web page into a parsable object
 - ▣ **html_nodes()** → Extracts elements with provided *tags*
 - ▣ **html_text()** → Gets the text associated with an element
- Scraping is easier if you *understand how the web works...*
 - ▣ Structure of HTML
 - ▣ Nature of HTTP requests
- Sometimes scraping needs to be *automated...*

Understanding how the web works



Google Search

I'm Feeling Lucky

Helpful tips to fact check information online

<https://www.google.com/search?q=Duke+University>

USGS Water Data for the Nation

Search for Sites With Data

Current
Conditions

Sites with real-time or recent surface-water, groundwater, or water-quality data.

Site Information

Descriptive site information for all sites with links to all available water data for individual sites.

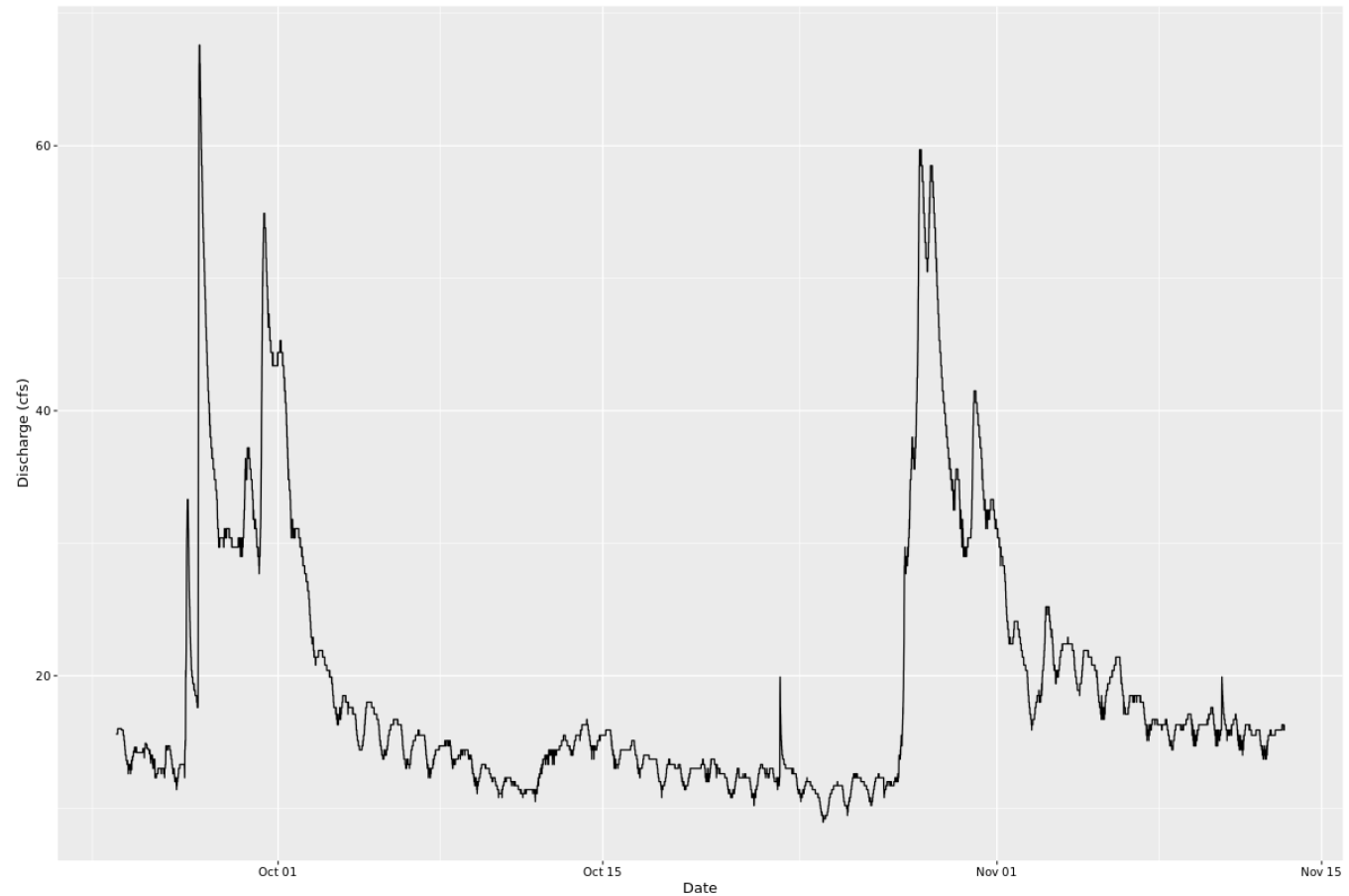


Map of all sites with links to all available water data for individual sites.

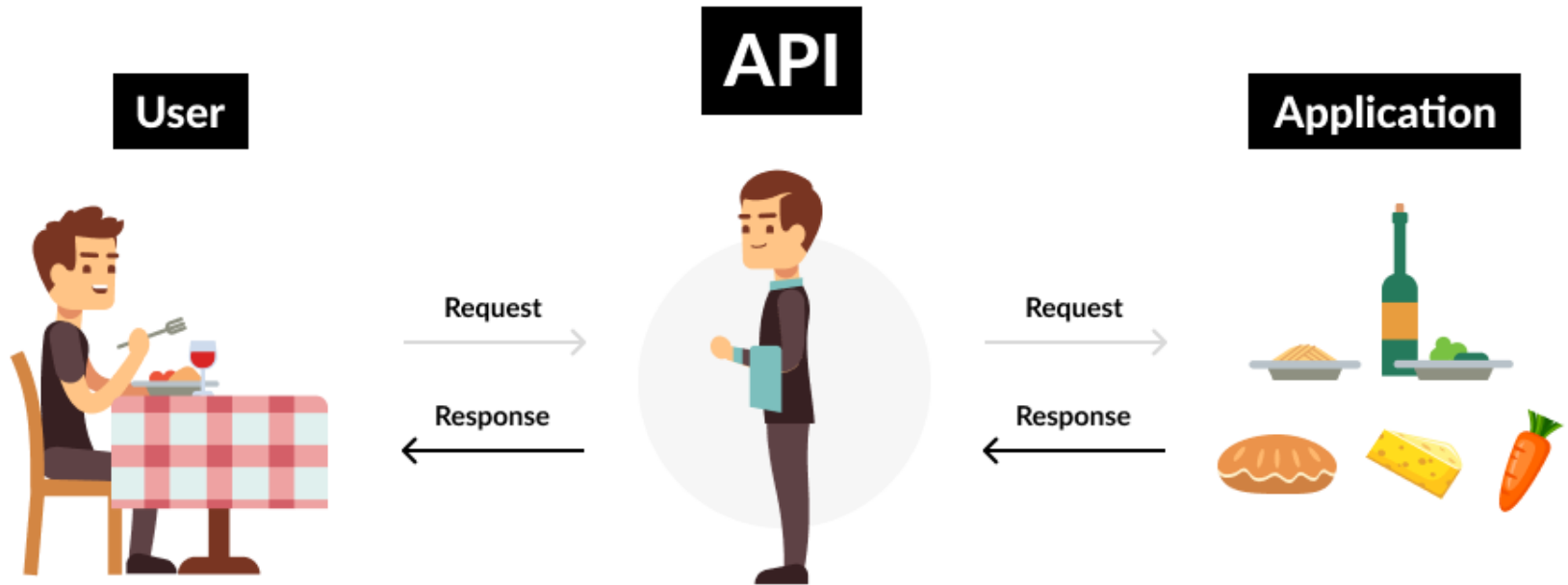
https://waterdata.usgs.gov/nwis/uv?search_site_no=02087500&period=7&format=rdb

Exercise: Pull gage data from NWIS

Last 24 days
from Gage #
02085070



APIs



Scraping data

1. Find tags with Selector Gadget
2. Scrape data into R:
 - `read_html()` → `html_nodes()` → `html_text()`
3. Construct a dataframe from scraped data...
4. Analyze data...

Automating the scraping process

1. Analyze the URL and identify tags
2. Create a function to scrape data
 - Make variables for building URL, tags
3. Call function via ``lapply`` or *Purrr's* ``map`` function

Alternatives...

- APIs & Packages
 - ▣ Census (via “tidycensus” package)
 - ▣ USGS (via “dataRetrieval” package)



Solutions

Scrape & Plot NWIS Data

```
```{r EXERCISE - pull.and.plot.other.discharge.data}
#Set the URL
theURL <-
 'https://waterdata.usgs.gov/nwis/uv?search_site_no=02085070&period=24&format=rdb'

#Get the data, which starts on line 27
gage_data <- read.table(
 theURL,
 skip = 26,
 header=TRUE,
 sep='\t',
 stringsAsFactors = T)

#Update the column headers
colnames(gage_data) = c(
 "agency_cd", "site_no", "datetime", "tz_cd",
 "discharge_cfs", "89192_00060_cd", "gage_ht_ft", "89193_00065_cd")

#Tidy the data
gage_data <- gage_data %>%
 select(datetime, discharge_cfs, gage_ht_ft) %>%
 mutate(datetime = ymd_hm(datetime))

#Plot
ggplot(gage_data,aes(x=datetime,y=discharge_cfs)) +
 geom_line() +
 labs(x = "Date",y = "Discharge (cfs)")
```
```

Scrape Data from EIA site

```
```{r EXERCISE - scrape.data.manually eval=FALSE}
#1 Link to the web site using read_html
the_website <- read_html('https://www.eia.gov/electricity/state/')

#2&3 Locate elements and read their text attributes into variables
the_states <- the_website %>% html_nodes('td:nth-child(1)') %>% html_text()
the_price <- the_website %>% html_nodes('td:nth-child(2)') %>% html_text()
the_capacity <- the_website %>% html_nodes('td:nth-child(3)') %>% html_text()
net_generation <- the_website %>% html_nodes('td:nth-child(4)') %>% html_text()
total_retail <- the_website %>% html_nodes('td:nth-child(5)') %>% html_text()

#3 Construct a dataframe from the values
energy_data <- data.frame(
 "State" = the_states,
 "Price" = as.numeric(the_price),
 "Capacity" = as.numeric(gsub(',', '', the_capacity)),
 "Net Generation" = as.numeric(gsub(',', '', the_capacity)),
 "Total Retail" = as.numeric(gsub(',', '', total_retail))
) %>%
 filter(State != 'U.S. Total')|
```
```

Scrape as a function

```
```{r create.scrape.function}
scrape.it <- function(the_year){
 #Get the proper url
 the_url <- ifelse(
 the_year=='2024',
 'https://www.eia.gov/electricity/state/',
 paste0('https://www.eia.gov/electricity/state/archive/',the_year, '/')
)

 #Fetch the website
 the_website <- read_html(the_url)

 #Scrape the data
 the_states <- the_website %>% html_nodes('td:nth-child(1)') %>% html_text()
 the_price <- the_website %>% html_nodes('td:nth-child(2)') %>% html_text()
 the_capacity <- the_website %>% html_nodes('td:nth-child(3)') %>% html_text()
 net_generation <- the_website %>% html_nodes('td:nth-child(4)') %>% html_text()
 total_retail <- the_website %>% html_nodes('td:nth-child(5)') %>% html_text()

 #Convert to dataframe
 energy_data <- data.frame(
 "State" = the_states,
 "Price" = as.numeric(the_price),
 "Capacity" = as.numeric(gsub(',', '', the_capacity)),
 "Net Generation" = as.numeric(gsub(',', '', the_capacity)),
 "Total Retail" = as.numeric(gsub(',', '', total_retail))
) %>%
 filter(State != 'U.S. Total') %>%
 mutate(Year = the_year)

 #Return the dataframe
 return(energy_data)
}
```

# Applying scrape function across years

```
Use the above function to scrape data for 2017
energy_2017 <- scrape.it(2017)

Map the function to scrape data from 2017 to 2024
energy_17_24 <- seq(2017,2024) %>% map(scrape.it) %>% bind_rows()

Example plot
energy_17_24 %>%
 filter(State %in% c('North Carolina','South Carolina', 'Virginia')) %>%
 ggplot(aes(x=Year, y=Price, color=State)) +
 #geom_line() +
 geom_smooth(method='loess',se=FALSE) +
 scale_x_date(date_breaks = '1 year', date_labels = '%Y')
```

